

Python: datos compuestos y condicionales

Listas, tuplas, diccionarios, conjuntos y toma de decisiones

1. Datos simples y datos compuestos

Un dato simple representa un único valor: un número entero, un decimal, un texto, un booleano o None. Un dato compuesto permite reunir varios valores dentro de una misma estructura. Esto es necesario cuando queremos trabajar con colecciones de datos: notas de un alumno, productos de una compra, coordenadas, fichas de usuarios o configuraciones de un programa.

Los datos compuestos más importantes en una primera etapa de Python son las listas, las tuplas, los diccionarios y los conjuntos. Cada uno organiza la información de una forma distinta.

Tipo	Símbolo	Idea principal	Ejemplo
list	[]	Varios valores ordenados y modificables.	notas = [7, 8, 9]
tuple	()	Varios valores ordenados que no se modifican.	punto = (10, 20)
dict	{}	Datos guardados mediante claves.	alumno = {"nombre": "Ana"}
set	{}	Colección de valores únicos.	colores = {"rojo", "azul"}

La pregunta clave no es solo cuántos datos quiero guardar, sino cómo quiero acceder a ellos después.

2. Listas

Una lista es una estructura que permite guardar varios valores en un orden concreto. Los elementos se escriben entre corchetes y se separan con comas. Es uno de los tipos compuestos más utilizados en Python.

```
notas = [7, 8, 9]
nombres = ["Ana", "Luis", "Marta"]
mezcla = [10, "Python", True]
```

Una lista puede contener elementos de distintos tipos, aunque en la práctica suele ser más claro que todos los elementos representen el mismo tipo de información.

Acceso por posición

Las listas están ordenadas. Cada elemento tiene una posición, también llamada índice. En Python, el primer índice es 0.

```
notas = [7, 8, 9]

primera = notas[0]
segunda = notas[1]
tercera = notas[2]
```

Si una lista tiene tres elementos, sus índices son 0, 1 y 2. Intentar acceder a notas[3] daría error.

3. Modificar listas

Las listas son modificables. Esto significa que podemos cambiar, añadir o eliminar elementos después de haberlas creado.

Cambiar un elemento

```
notas = [7, 8, 9]
notas[1] = 10

print(notas)
```

La lista pasa de [7, 8, 9] a [7, 10, 9]. El elemento situado en la posición 1 cambia de valor.

Añadir elementos

```
nombres = ["Ana", "Luis"]
nombres.append("Marta")

print(nombres)
```

El método `append` añade un elemento al final de la lista.

Longitud de una lista

```
notas = [7, 8, 9]
cantidad = len(notas)

print(cantidad)
```

`len` devuelve cuántos elementos hay en la lista. En este caso, el resultado es 3.

4. Tuplas

Una tupla se parece a una lista porque también guarda varios valores ordenados. La diferencia principal es que una tupla no se puede modificar una vez creada. Por eso se dice que es inmutable.

```
coordenada = (10, 20)
fecha = (2026, 9, 22)
color_rgb = (255, 0, 0)
```

Las tuplas se usan cuando queremos representar un grupo de valores que tiene sentido como unidad y que no debería cambiar.

Acceso a elementos

```
coordenada = (10, 20)

x = coordenada[0]
y = coordenada[1]
```

El acceso por índice funciona igual que en las listas: el primer elemento está en la posición 0.

Lista	Tupla
Se escribe con corchetes: [1, 2, 3]	Se escribe con paréntesis: (1, 2, 3)
Se puede modificar.	No se puede modificar.
Útil para colecciones que cambian.	Útil para datos fijos o agrupaciones estables.

5. Diccionarios

Un diccionario guarda datos mediante pares clave-valor. En una lista accedemos a los datos por posición; en un diccionario accedemos por clave. Esto hace que los diccionarios sean muy útiles para representar objetos, fichas o registros.

En muchos ejemplos de iniciación las claves son textos, porque así el código resulta más legible: nombre, edad, curso o nota. Sin embargo, en Python las claves no tienen que ser siempre strings. También pueden ser números, booleanos o tuplas, siempre que sean valores que Python pueda usar como clave. Para empezar, lo más recomendable es utilizar strings como claves.

```
alumno = {  
    "nombre": "Ana",  
    "edad": 16,  
    "curso": "IA"  
}
```

En este ejemplo, nombre, edad y curso son claves. Ana, 16 e IA son los valores asociados a esas claves.

Acceder a un valor

```
alumno = {  
    "nombre": "Ana",  
    "edad": 16  
}  
  
nombre = alumno["nombre"]  
edad = alumno["edad"]
```

Un diccionario se entiende muy bien si lo imaginamos como una ficha: cada campo tiene un nombre y un valor.

6. Modificar diccionarios

Los diccionarios son modificables. Podemos cambiar el valor de una clave, añadir nuevas claves o consultar cuántos pares clave-valor contiene.

Cambiar un valor

```
alumno = {  
    "nombre": "Ana",  
    "edad": 16  
}  
  
alumno["edad"] = 17
```

Añadir una clave nueva

```
alumno = {  
    "nombre": "Ana",  
    "edad": 16  
}  
  
alumno["nota"] = 8.5
```

Si la clave ya existe, se modifica su valor. Si la clave no existe, se crea una nueva entrada en el diccionario.

Operación	Ejemplo	Resultado
Consultar	alumno["nombre"]	Devuelve el valor asociado a la clave nombre.
Modificar	alumno["edad"] = 17	Cambia el valor de edad.
Añadir	alumno["nota"] = 8.5	Crea una nueva clave llamada nota.
Longitud	len(alumno)	Devuelve el número de claves.

7. Conjuntos

Un conjunto, o set, guarda varios valores sin repetir. Si intentamos guardar el mismo valor varias veces, Python conserva una sola aparición. Los conjuntos no se usan para acceder por posición, sino para trabajar con pertenencia y valores únicos.

```
colores = {"rojo", "azul", "rojo"}  
  
print(colores)
```

Aunque escribimos rojo dos veces, el conjunto solo lo conserva una vez.

Cuándo son útiles

- Cuando queremos eliminar duplicados.
- Cuando queremos saber si un valor pertenece a una colección.
- Cuando no nos importa el orden de los elementos.

```
usuarios = {"ana", "luis", "marta"}  
  
existe = "ana" in usuarios
```

En una primera clase basta con entender que un set guarda valores únicos. Las operaciones de teoría de conjuntos pueden verse más adelante.

8. Elegir el tipo compuesto adecuado

No todos los datos compuestos sirven para lo mismo. Elegir bien la estructura ayuda a escribir programas más claros. Una forma sencilla de decidir es preguntarse cómo necesitamos guardar y recuperar la información.

Pregunta	Tipo recomendado	Ejemplo
¿Necesito una secuencia ordenada que pueda cambiar?	list	Lista de notas.
¿Necesito varios valores fijos?	tuple	Coordenadas de un punto.
¿Necesito datos con nombre de campo?	dict	Ficha de un alumno.
¿Necesito valores únicos sin duplicados?	set	Usuarios registrados.

La decisión no depende solo del número de datos. Depende de la relación entre ellos y de cómo queremos usarlos.

```
alumno = {  
    "nombre": "Ana",  
    "notas": [7, 8, 9],  
    "activo": True  
}
```

Los datos compuestos también pueden combinarse. En este ejemplo, el diccionario guarda un texto, una lista y un booleano.

9. Condicionales: tomar decisiones

Un condicional permite que un programa ejecute unas instrucciones u otras según se cumpla una condición. La condición siempre se evalúa como verdadera o falsa.

```
edad = 18

if edad >= 18:
    print("Mayor de edad")
```

La línea del if termina con dos puntos. Las instrucciones que dependen del if aparecen indentadas, es decir, desplazadas hacia la derecha.

La indentación no es decorativa en Python: indica qué instrucciones pertenecen al bloque del if.

Condiciones

Una condición suele construirse con operadores de comparación.

Operador	Significado	Ejemplo
==	igual que	edad == 18
!=	distinto de	nombre != "Ana"
>	mayor que	nota > 5
>=	mayor o igual que	edad >= 18
<	menor que	precio < 100
<=	menor o igual que	nota <= 10

10. if y else

La estructura if permite ejecutar un bloque si la condición es verdadera. La estructura else permite indicar qué debe ocurrir cuando la condición no se cumple.

```
edad = 16

if edad >= 18:
    print("Puede entrar")
else:
    print("No puede entrar")
```

Si edad >= 18 es True, se ejecuta el primer bloque. Si es False, se ejecuta el bloque del else.

Errores frecuentes

- Olvidar los dos puntos al final de la línea del if o del else.
- No indentar las instrucciones que pertenecen al condicional.
- Confundir = con ==. Un solo igual asigna; dos iguales comparan.

```
nota = 7

if nota >= 5:
    resultado = "aprobado"
else:
    resultado = "suspense"

print(resultado)
```

11. if, elif y else

Cuando hay más de dos posibilidades, se utiliza elif. En otros lenguajes se suele escribir else if, pero en Python se escribe elif.

```
nota = 7

if nota < 5:
    print("Suspenso")
elif nota < 7:
    print("Aprobado")
elif nota < 9:
    print("Notable")
else:
    print("Sobresaliente")
```

Python evalúa las condiciones de arriba hacia abajo. Cuando encuentra una condición verdadera, ejecuta su bloque y ya no revisa el resto de opciones del mismo condicional.

El orden de las condiciones importa. No es lo mismo empezar preguntando `nota < 5` que empezar preguntando `nota < 9`.

```
temperatura = 30

if temperatura < 10:
    print("Hace frío")
elif temperatura < 25:
    print("Temperatura agradable")
else:
    print("Hace calor")
```

12. Condiciones compuestas

A veces una decisión depende de más de una condición. Para combinar condiciones usamos los operadores lógicos and, or y not.

Operador	Significado	Cuándo da True
and	y	Cuando todas las condiciones son True.
or	o	Cuando al menos una condición es True.
not	no	Invierte el valor lógico.

```
edad = 17
tiene_permiso = True

if edad >= 16 and tiene_permiso:
    print("Puede participar")
else:
    print("No puede participar")
```

En este ejemplo deben cumplirse las dos condiciones: tener al menos 16 años y tener permiso.

```
es_profesor = False
es_administrador = True

if es_profesor or es_administrador:
    print("Puede acceder")
```

Aquí basta con que una de las condiciones sea verdadera.

13. Condicionales anidados

Un condicional anidado es un if dentro de otro if. Puede ser útil, pero si se abusa de él el código se vuelve difícil de leer.

```
edad = 20
tiene_entrada = True

if edad >= 18:
    if tiene_entrada:
        print("Puede entrar")
    else:
        print("Necesita entrada")
else:
    print("No tiene edad suficiente")
```

El segundo if solo se evalúa si el primero se cumple. Por eso la indentación crea varios niveles de decisión.

Alternativa con and

```
edad = 20
tiene_entrada = True

if edad >= 18 and tiene_entrada:
    print("Puede entrar")
else:
    print("No puede entrar")
```

Cuando las condiciones son sencillas, combinarlas con and puede hacer el programa más fácil de leer.

14. Condicional ternario

El condicional ternario permite asignar un valor u otro en una sola línea. No sustituye al if normal, pero resulta útil cuando la decisión es muy breve.

```
edad = 20

mensaje = "Mayor de edad" if edad >= 18 else "Menor de edad"

print(mensaje)
```

La estructura general es:

```
valor_si_true if condicion else valor_si_false
```

Conviene usarlo solo cuando mejora la claridad. Si la condición es larga o hay varias instrucciones, es preferible escribir un if normal.

Forma normal	Forma ternaria
<pre>if edad >= 18: mensaje = "Mayor" else: mensaje = "Menor"</pre>	<pre>mensaje = "Mayor" if edad >= 18 else "Menor"</pre>

15. match-case: alternativa al switch

Python no utiliza un switch clásico como otros lenguajes. Desde Python 3.10 existe match-case, que permite comparar un valor con diferentes casos. Para una primera clase, basta con conocerlo como una ampliación.

```
dia = "lunes"

match dia:
    case "lunes":
        print("Inicio de semana")
    case "viernes":
        print("Casi fin de semana")
    case _:
        print("Otro día")
```

El caso `_` funciona como opción por defecto: se ejecuta cuando no coincide ningún caso anterior.

Para empezar, if, elif y else son más importantes. match-case puede presentarse como el equivalente moderno al switch.